

Universidade Federal do Rio Grande do Norte
Escola de Ciências e Tecnologia
IX Semana de Ciências e Tecnologia
Minicurso de Introdução a Banco de Dados

SQL e PostgreSQL

Parte 02

Breno Santana Santos
⟨breno.santos@ufrn.br⟩

10 de novembro de 2026

Sumário

1 Gerenciamento de Banco de Dados e Tabelas
- Linguagem DML

- Consideremos as seguintes tabelas:

```
CREATE TABLE tb_funcionario (
  matricula SERIAL NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  salario NUMERIC(10,2) NOT NULL,
  cod_dep INT NULL,
  ativo BOOL
);

CREATE TABLE tb_departamento (
  codigo INT NOT NULL,
  nome VARCHAR(50) NOT NULL
)
```

- Inserção de Dados:
  - Permite inserir novos registros em uma tabela.
  - **Sintaxe:** `INSERT INTO tb_nome (colunas_sep_vírgula) VALUES (valores_sep_vírgula);`
  - **Exemplo:**
    - `INSERT INTO tb_funcionario (nome, salario, cod_dep, ativo) VALUES ('Fulano de Tal', 1000, 1, true);`
    - `INSERT INTO tb_departamento (codigo, nome) VALUES (1, 'Contabilidade');`
  - Não precisa informar os valores para os campos autoincrementais.
  - O comando `INSERT` é posicional, ou seja, deve-se respeitar a ordem dos atributos especificados para a especificação dos valores desses atributos no comando.
  - A lista de colunas pode ser omitida, sendo necessário informar todos os valores na ordem em que as colunas foram definidas na tabela.

- Inserção de Dados (cont.):
  - Se a lista de colunas for omitida e existir uma coluna de propriedade autoincremental na tabela, devemos informar o valor NULL para a coluna autoincremental.
  - **Exemplo:**
    - `INSERT INTO tb_funcionario (nome, salario, cod_dep, ativo) VALUES ('Fulano de Tal', 1000, 1, 1);`
    - `INSERT INTO tb_funcionario VALUES (NULL, 'Fulano de Tal', 1000, 1, 1);`
- Atualização de Dados:
  - Permite modificar valores em uma tabela.
  - **Sintaxe:** `UPDATE tb_nome SET coluna = valor [, ... coluna = valor] [WHERE condição];`

- Atualização de Dados (cont.):
  - **Exemplo:**
    - `UPDATE tb_funcionario SET nome = 'Cicrano de Tal' WHERE matricula = 1;`
    - `UPDATE tb_funcionario SET nome = 'Cicrano de Tal', cod_dep = 2 WHERE matricula = 1;`
    - `UPDATE tb_departamento SET codigo = 2 WHERE codigo = 1;`
  - A cláusula WHERE representa os filtros de registros por meio de condicionais para comandos SQL.
  - Quando não for especificado uma condição, a atualização ocorrerá em todos os registros.
  - **Exemplo:** `UPDATE tb_funcionario SET nome = 'Cicrano de Tal', cod_dep = 3;`

- Remoção de Dados:
  - Permite remover linhas (registros) de uma tabela.
  - **Sintaxe:** `DELETE FROM tb_nome [WHERE condição];`
  - **Exemplo:**
    - `DELETE FROM tb_funcionario WHERE matricula = 1;`
    - `DELETE FROM tb_departamento WHERE codigo = 1;`
  - Quando não for especificado uma condição, serão removidos todos os registros.
  - **Exemplo:**
    - `DELETE FROM tb_funcionario;`
    - `DELETE FROM tb_departamento;`

- Recuperação de Dados:
  - Permite retornar um conjunto de linhas (registros) de uma ou mais tabelas. Busca informações em uma tabela e retorna o resultado da busca.
  - **Sintaxe:** `SELECT <lista_colunas_sep_vírgula> FROM <lista_tabelas_sep_vírgula> WHERE <condição>;`
  - **Exemplo:** `SELECT matricula, nome FROM tb_funcionario WHERE cod_dep = 1;`
  - Quando todas as colunas devem ser recuperadas, é mais prático utilizar o caractere *, pois representa todas as colunas em um comando SELECT.
  - **Exemplo:** `SELECT * FROM tb_funcionario;`

- Recuperação de Dados (cont.):
  - Podemos recuperar colunas de várias tabelas, de modo que os registros das tabelas consultadas serão "cruzados".
  - **Exemplo:** `SELECT * FROM tb_funcionario, tb_departamento;`
  - Quando duas tabelas possuem colunas com mesmo nome, devemos utilizar o nome da tabela para eliminar a ambiguidade.
  - **Exemplo:** `SELECT tb_funcionario.nome, tb_departamento.nome FROM tb_funcionario, tb_departamento;`
  - As tabelas e colunas de uma consulta podem ser "apelidadas", ou seja, um nome associado a um atributo ou tabela através do operador AS (alias).
  - Pode-se omitir o operador AS. Devemos ter cuidado com isso.

- Recuperação de Dados (cont.):
  - Quando o alias possuir espaço em branco, usa-se as aspas simples.
  - **Exemplo:**
    - `SELECT f.* FROM tb_funcionario AS f;`
    - `SELECT f.nome AS 'nome do funcionário' FROM tb_funcionario AS f;`
  - A cláusula DISTINCT pode ser utilizada para remover linhas (registros) repetidas.
  - **Exemplo:**
    - `SELECT DISTINCT f.nome FROM tb_funcionario f;`
    - `SELECT DISTINCT d.nome FROM tb_departamento d;`

- Recuperação de Dados (cont.):
  - Os principais operadores aritméticos do PostgreSQL são:
    - **Soma (+):** `SELECT (salario + 500) FROM tb_funcionario WHERE matricula >= (10 + 2);`
    - **Diferença (-):** `SELECT (salario - 500) FROM tb_funcionario WHERE matricula >= (10 - 2);`
    - **Multiplicação (*):** `SELECT (salario * 10) FROM tb_funcionario WHERE matricula >= (5 * 2);`
    - **Divisão (/):** `SELECT (salario / 5) FROM tb_funcionario WHERE matricula >= (20 / 4);`
  - Os principais operadores e funções de comparação do PostgreSQL que podem ser utilizados na cláusula WHERE são:
    - **Igualdade (=):** `SELECT * FROM tb_funcionario WHERE matricula = 2;`

- Recuperação de Dados (cont.):
  - Os principais operadores e funções de comparação do PostgreSQL que podem ser utilizados na cláusula WHERE são:
    - **Desigualdade (<> ou !=):** `SELECT * FROM tb_funcionario WHERE matricula <> 2;`
    - **Menor ou Igual a (<=):** `SELECT * FROM tb_funcionario WHERE matricula <= 10;`
    - **Menor ou Igual a (<):** `SELECT * FROM tb_funcionario WHERE matricula < 10;`
    - **Maior ou Igual a (>=):** `SELECT * FROM tb_departamento WHERE codigo >= 5;`
    - **Maior que (>):** `SELECT * FROM tb_funcionario WHERE matricula > 10;`
    - **Igualdade para valores não-alfanuméricos (IS):** `SELECT * FROM tb_funcionario WHERE ativo IS true;`

- Recuperação de Dados (cont.):
  - Os principais operadores e funções de comparação do PostgreSQL que podem ser utilizados na cláusula WHERE são:
    - **Desigualdade para valores não-alfanuméricos (IS NOT):** `SELECT * FROM tb_funcionario WHERE ativo IS NOT true;`
    - **Igualdade para valores nulos (IS NULL):** `SELECT * FROM tb_funcionario WHERE nome IS NULL;`
    - **Desigualdade para valores nulos (IS NOT NULL):** `SELECT * FROM tb_funcionario WHERE nome IS NOT NULL;`
    - **Confirmação de valor pertencente a um intervalo (BETWEEN...AND...):** `SELECT * FROM tb_funcionario WHERE matricula BETWEEN 5 AND 10;`
    - **Negação de valor pertencente a um intervalo (NOT BETWEEN...AND...):** `SELECT * FROM tb_funcionario WHERE matricula NOT BETWEEN 5 AND 10;`

- Recuperação de Dados (cont.):
  - Os principais operadores e funções de comparação do PostgreSQL que podem ser utilizados na cláusula WHERE são:
    - **Confirmação da existência de um padrão específico ou substring na coluna (LIKE):** `SELECT * FROM tb_funcionario WHERE nome LIKE 'a%';`
    - **Negação da existência de um padrão específico ou substring na coluna (NOT LIKE):** `SELECT * FROM tb_funcionario WHERE nome NOT LIKE 'a%';`

Observação

No uso dos operadores LIKE e NOT LIKE, existem alguns caracteres especiais que auxiliam na busca de padrões em cadeias de caracteres. São eles:

- **Percentagem (%):** ignora os valores antes e/ou depois desse caractere em uma *string* (cadeia de caracteres);
- **Underscore (_):** ignora um valor de caractere representado pelo underscore em uma string;
- **Barra Invertida (\):** representa o caractere de escape.

Exemplo

- `SELECT * FROM tb_funcionario WHERE nome LIKE '%ana%';`
- `SELECT * FROM tb_funcionario WHERE nome LIKE 'a_a';`
- `SELECT * FROM tb_funcionario WHERE observacoes LIKE '%100\%';`

- Recuperação de Dados (cont.):
  - Os principais operadores e funções de comparação do PostgreSQL que podem ser utilizados na cláusula WHERE são:
    - **Igualdade com múltiplos valores (IN):** `SELECT * FROM tb_funcionario WHERE matricula IN (2, 4, 6, 8, 10);`
    - **Desigualdade com múltiplos valores (NOT IN):** `SELECT * FROM tb_funcionario WHERE matricula NOT IN (2, 4, 6, 8, 10);`

Explicação

Os operadores IN e NOT IN verificam se um valor de atributo está contido ou não em um conjunto de valores.

- Recuperação de Dados (cont.):
  - Os principais operadores lógicos do PostgreSQL que podem ser utilizados na cláusula WHERE são:
    - **Negação (NOT ou !):** `SELECT * FROM tb_funcionario WHERE NOT matricula = 2;`
    - **Conjunção (AND ou &&):** `SELECT * FROM tb_funcionario WHERE matricula <> 2 AND nome NOT LIKE 'A%';`
    - **Disjunção (OR ou ||):** `SELECT * FROM tb_funcionario WHERE matricula <> 2 OR nome NOT LIKE 'A%';`
    - **Disjunção Exclusiva (XOR):** `SELECT * FROM tb_funcionario WHERE matricula <> 2 XOR nome NOT LIKE 'A%';`

- Recuperação de Dados (cont.):
  - Os resultados de uma consulta podem ser ordenados por meio da cláusula ORDER BY. É necessário indicar as colunas que serão utilizadas na ordenação.
  - **Exemplo:**
    - `SELECT * FROM tb_funcionario ORDER BY nome;`
    - `SELECT * FROM tb_departamento ORDER BY codigo;`
  - Quando dois ou mais registros possuem o mesmo valor para a coluna utilizada para ordenação, podemos utilizar outras colunas como "critério de desempate".
  - **Exemplo:**
    - `SELECT * FROM tb_funcionario ORDER BY nome, cargo, salario, cod_dep;`
    - `SELECT * FROM tb_departamento ORDER BY nome, codigo;`

- Recuperação de Dados (cont.):
  - Por padrão, a ordenação é de forma crescente, isto é, quando não se especifica o tipo de ordenação. A ordenação pode ser de forma decrescente com o uso do comando DESC após o nome da coluna. Caso deseje, informar que é crescente, utiliza-se o comando ASC após o nome da coluna.
  - **Exemplo:**
    - `SELECT * FROM tb_funcionario ORDER BY nome, cargo ASC, salario DESC, cod_dep;`
    - `SELECT * FROM tb_departamento ORDER BY nome ASC, codigo DESC;`

- Recuperação de Dados (cont.):
  - A quantidade de resultados de uma consulta pode ser especificada por meio da cláusula LIMIT. Se a quantidade de registros for inferior ao valor que for especificado na cláusula LIMIT, todos os registros serão retornados.
  - **Exemplo:** `SELECT * FROM tb_funcionario LIMIT 10;`
  - Podem ser desconsiderados os primeiros registros de uma consulta. Para isso, basta especificar dois parâmetros na cláusula LIMIT: os n-primeiros resultados que serão desconsiderados e a quantidade de registros a serem retornados, separando-os por vírgula.
  - **Exemplo:** `SELECT * FROM tb_funcionario LIMIT 5, 10;`

- Recuperação de Dados (cont.):
  - Quando é necessário que duas ou mais tabelas sejam unidas ou combinadas, aumentando o número de registros, podemos utilizar o operador UNION ou UNION ALL. Devemos especificar as colunas a serem utilizadas. Para cada SELECT, a quantidade de colunas deve ser a mesma, bem como seus tipos de dados devem ser os mesmos.
  - O operador UNION seleciona os registros distintos (sem repetição).
  - O operador UNION ALL seleciona todos os registros, mesmo havendo duplicidade, ou seja, registros repetidos.
  - **Exemplo:**
    - ```
      SELECT matricula, nome FROM tb_funcionario
      UNION
      SELECT codigo, nome FROM tb_departamento;
      ```
    - ```
      SELECT matricula, nome FROM tb_funcionario
      UNION ALL
      SELECT codigo, nome FROM tb_departamento;
      ```